COMPASS_Synoptic_LE_Tree_CH4_Flux_Calcs_2022

Roberta Peixoto

2026-04-27

Table of contents

1 Add Required Packages

2 Pull in Raw Datafiles

```
#defining where the raw data is

DATA_DIR_ROOT <- here::here(
  "Tree_Fluxes_Synoptic_LE",
  "2022",
  "Raw Data"
)


#fucntion to read data
now <- function() format(Sys.time(), "%Y-%m-%d %H:%M:%S")
message(now(), " | Script started")
#finding the zip files
message(now(), " | Looking for ZIP files in: ", DATA_DIR_ROOT)

zip_files <- list.files(
  DATA_DIR_ROOT,
  pattern = "\\.zip$",
  full.names = TRUE)

message(now(), " | ZIP files found: ", length(zip_files))
```

```
print(zip_files)
  if (length(zip_files) == 0) {
  stop("No ZIP files found. Check DATA_DIR_ROOT.")}

cache <- list()

# Reader function
read_li7810_zip <- function(zip_file, tz = "US/Eastern") {
  message(now(), " | Processing ZIP: ", basename(zip_file))
  if (zip_file %in% names(cache)) {
    message("  -> Using cached version")
    return(cache[[zip_file]])  }

  tmp_dir <- tempfile("li7810_")
  dir.create(tmp_dir)
  on.exit(unlink(tmp_dir, recursive = TRUE), add = TRUE)
  unzip(zip_file, exdir = tmp_dir)
  data_files <- list.files(
    tmp_dir,
    pattern = "\\.data$",
    full.names = TRUE,
    recursive = TRUE  )

  message("  -> .data files found: ", length(data_files))
  if (length(data_files) == 0) {
    stop("No .data files inside ZIP: ", zip_file) }

  dat_list <- lapply(data_files, function(f) {
    message("     Reading: ", basename(f))
    ffi_read_LI7810(f)})

  dat <- data.table::rbindlist(dat_list, fill = TRUE) %>%
    mutate(TIMESTAMP = with_tz(TIMESTAMP, tz))
  cache[[zip_file]] <<- dat
  message("  -> ZIP finished: ", basename(zip_file))
  return(dat)}

# ACTUAL EXECUTION
message(now(), " | Starting data read")
alldat_list <- lapply(zip_files, read_li7810_zip)
alldat <- data.table::rbindlist(alldat_list, fill = TRUE)
message(now(), " | DONE")
```

```
message("Final number of rows: ", nrow(alldat))

# Show result
print(head(alldat))

#REMOVING WEIRD VERY LOW VALUES ON "2022-08-17 UP 7 148" cH4=-8000 licor bug???
alldat<-alldat%>%filter(CH4>-1000)
```

3 Read in metadata file and match to raw data

```
md <- "Raw Data/COMPASS_SynopticLE_TreeFlux_Metadata_2022b.csv"
dat <- read.csv(md)

metadat <- dat %>%
  # Remove rows without date
  filter(!is.na(Date)) %>%
  # Keep only desired instruments
  filter(SN %in% c("TG10-01448", "TG10-01426", "TG10-01028")) %>%
  # Normalize Plot labels
  mutate(
    Plot_raw = toupper(Plot),
    # Extract plot components
    #Site = str_extract(Plot_raw, "^[A-Z]{3}"),
    #Zone = str_extract(Plot_raw, "(TR|UP)"), # somehow all of the UP PTR measurements turne
    Site = toupper(Site),
    Zone = toupper(Zone),
    Replicate = str_extract(Plot_raw, "(?<=TR|UP)[0-9]+"),
    # Letter ONLY if it comes AFTER a number
    Letter = str_extract(Plot_raw, "(?<=[0-9])[A-Z]$"),
    # Time normalization
    Time_start = str_replace_all(as.character(Time_start), "\\.", ":") |> str_trim(),
    Stop.time  = str_replace_all(as.character(Stop.time),  "\\.", ":") |> str_trim(),
    # Safe time parsing (NO WARNINGS)
    t_start = suppressWarnings(parse_date_time(Time_start, orders = c("H:M:S", "H:M"))),
    t_stop  = suppressWarnings(parse_date_time(Stop.time,  orders = c("H:M:S", "H:M"))),
    # Observation length
    Obs_length = as.numeric(t_stop - t_start, units = "secs"),
    # Timestamp
    TIMESTAMP = as.POSIXct(
```

```
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    # Sequential ID
    Seq = row_number(),
    # INAL standardized Plot label
    Plot = if_else(
      is.na(Letter),
      paste(Date, Site, Zone, Replicate, Seq, sep = "_"),
      paste(Date, Site, Zone, Replicate, Letter, Seq, sep = "_")) )

# Quick check
head(metadat$Plot)
```

4 Check start times overlapsor missing values

```
#checking if there is any start and stop time overlap from the field notes
overlaps_detected <- metadat %>%
  mutate(
    start_ts = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    end_ts = start_ts + Obs_length
  ) %>%
  arrange(SN, start_ts) %>%
  group_by(SN) %>%
  mutate(
    next_plot     = lead(Plot),
    next_start_ts = lead(start_ts),
    overlap_ffi   = !is.na(next_start_ts) & next_start_ts <= end_ts
  ) %>%
  filter(overlap_ffi) %>%
  select(
    SN,
    Plot,
    start_ts,
```

```
    end_ts,
    next_plot,
    next_start_ts,
    Obs_length
  )

print(overlaps_detected)

#checking problems with obs_length - this is a frequent bug for manual field notes
field_notes_overlap<-metadat %>%
  mutate(
    obs_problem =
      is.na(Obs_length) | Obs_length <= 0
  ) %>%
  filter(obs_problem) %>%
  select(
    SN,
    Date,
    Plot,
    Time_start,
    Stop.time,
    Obs_length
  )

print(field_notes_overlap)

# checking unique measurment start time and stop time
# alldat %>%
#     filter(
#         SN == "TG10-01448",
#         TIMESTAMP > as.POSIXct("2023-01-05 13:28:00"),
#         TIMESTAMP < as.POSIXct("2023-01-05 13:33:00")
#     ) %>%
#     ggplot(aes(x = TIMESTAMP, y = CO2)) +
#     geom_point()#+ylim(300,500)
```

5 Match metadata to raw data

```
#function to match data per equipment (considering different serial numbers) uhul!
match_flux_metadata <- function(alldat, metadat) {
```

```
 message("Starting metadata  Licor matching")
 #Sanity checks (fail early)
 stopifnot(
   "TIMESTAMP" %in% names(alldat),
   "SN"        %in% names(alldat),
   "Date"      %in% names(metadat),
   "Time_start"%in% names(metadat),
   "Obs_length"%in% names(metadat),
   "Plot"      %in% names(metadat)
 )

 stopifnot(all(metadat$Obs_length > 0, na.rm = TRUE))
 stopifnot(anyDuplicated(metadat$Plot) == 0)
# Instruments present in BOTH datasets
 instruments <- intersect(
   unique(alldat$SN),
   unique(metadat$SN)
 )
 message("Instruments to process: ", paste(instruments, collapse = ", "))
 results <- lapply(instruments, function(sn) {
   message("Processing SN: ", sn)
   dat_i <- alldat %>%
     filter(SN == sn)
   meta_i <- metadat %>%
     filter(SN == sn) %>%
     arrange(Date, Time_start) %>%
     mutate(metadat_row = row_number())
   stopifnot(nrow(meta_i) > 0)
 # Actual matching
   dat_i$metadat_row <- ffi_metadata_match(
     data_timestamps = dat_i$TIMESTAMP,
     start_dates     = meta_i$Date,
     start_times     = meta_i$Time_start,
     obs_lengths     = meta_i$Obs_length  )
 # Attach metadata (vectorized, fast)
   dat_i <- dat_i %>%
     mutate(
       Plot      = meta_i$Plot[metadat_row],
       Volume    = meta_i$Volume[metadat_row],
       Temp      = meta_i$Temp[metadat_row],
       Area      = meta_i$Area[metadat_row],
       Dead_band = meta_i$Dead_band[metadat_row]      )
```

```
  # Drop unmatched rows
    n_before <- nrow(dat_i)
    dat_i <- dat_i %>% filter(!is.na(metadat_row))
    n_after <- nrow(dat_i)
    message("  Matched rows: ", n_after, " / ", n_before)
    dat_i })
  alldat_matched <- rbindlist(results, fill = TRUE)
  #Final sanity checks
  stopifnot(
    !anyNA(alldat_matched$Plot),
    nrow(alldat_matched) > 0
  )
  message("Matching finished successfully")
  return(alldat_matched)}

#running the funciton
message("Running final metadata matching...")
alldat_matched <- match_flux_metadata(alldat, metadat)
message("Matched dataset rows: ", nrow(alldat_matched))
head(alldat_matched)

#saving matched data and metadata as raw data is too big (today, Licor files leave in the COM
saveRDS(alldat_matched, file = "alldat_matched_2022.rds")

#can remove any row that have NA's in the CH4 column
#alldat <- alldat %>%
 # filter(!is.na(CH4))
```

6 reating alldata matched df

```
# after you match the data for the first time - we can just read the .rds file to save proces
# if you need to match licor data and metadata again, remove "#" from the previous chunks

alldat_matched <- readRDS("alldat_matched_2022.rds")
```

#Extracting initial CH4 concentration

```
## Extract initial CH4 concentration (t0) for each Plot for QAQC later
#saving df to use after flux calculation
```

```r
df_CH4_in <- alldat_matched %>%
  arrange(Plot, TIMESTAMP) %>%
  group_by(Plot) %>%
  summarise(CH4_initial = first(CH4), .groups = "drop")
```

7 Creating temperature csv from the L2-weather station in the wetland

```r
# Getting Air temp from the weather station
## Read L2 temperature data from ZIP stored outside GitHub,
## extract desired variable, save lightweight CSV,
## and remove temporary unzipped files afterwards

## Read extracted L2 Parquet files and save temperature time series
BASE_DIR <- "D:/COMPASS_L2-v2-1/v2-1"
SITE     <- "(OWC|CRC|PTR)"
VARIABLE <- "wx-tempavg15"

OUTFILE  <- "Raw Data/L2_temp_timeseries.csv"

# Build regex that matches REAL L2 filenames
regex <- paste0(SITE, ".*", VARIABLE,".*_L2_v2-1.*\\.parquet$")

# Locate Parquet files
files <- list.files(BASE_DIR,pattern = regex,recursive = TRUE,full.names = TRUE)

message("Files found: ", length(files))
print(basename(files))

# Fail fast if nothing is found
stopifnot(length(files) > 0)

# Read and combine Parquet files
dat <- lapply(files, function(f) {
  message("Reading ", basename(f))
  read_parquet(f)
})

dat <- bind_rows(dat)
```

```
# Optional QA filtering (only if column exists)
if ("QA_flag" %in% names(dat)) {
  dat <- dat %>% filter(QA_flag == 0)
}

# Keep only necessary columns
temps <- dat %>%
  select(TIMESTAMP, Plot, Site, Value) %>%
  rename(Temp = Value)

head(temps)

# Save lightweight output only
write.csv(
  temps,
  file = OUTFILE,
  row.names = FALSE
)
```

8 Reading Temp df to merge to metadat

```
#reading temp
temps <- read.csv("Raw Data/L2_temp_timeseries.csv")

## Match up the L1 temp data and add it to the metadata dataframe
#Going to need to use site and timestamp in the metadata in order to find a date

#make sure time stamps are in the correct format
alldat_matched$TIMESTAMP <- as.POSIXct(alldat_matched$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS
#problem here L2 data for midnight is always "2022-01-01" we loose this line to NA when forma
temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)] <-
  paste0(temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)], " 00:00:00")
temps$TIMESTAMP <- as.POSIXct(temps$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")

#time rounding step
alldat_matched$TIMESTAMP_round <- round_date(alldat_matched$TIMESTAMP, "60 mins")
temps$TIMESTAMP_round <- round_date(temps$TIMESTAMP, "60 mins")

#ensure there is only one data observation per hour
```

```
temps_clean <-   temps %>%
    group_by(TIMESTAMP_round, Site) %>%
    summarize(Temp = mean(Temp ))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by TIMESTAMP_round and Site.
i Output is grouped by TIMESTAMP_round.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(TIMESTAMP_round, Site))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
#join them
alldat_matched <- alldat_matched %>%
  left_join(temps_clean, by=c("Site", "TIMESTAMP_round"))
```

9 fixing some Deadbands and stop time

```
# need to include a way to adjust the stop time too
# Table of plots with custom deadband adjustments
time_overrides <- tibble::tibble(
            Plot = c("10/21/2022_OWC_TR_121_56",
                     "8/17/2022_OWC_UP_7_148",
                    "10/7/2022_PTR_TR_7_93",
                    "10/4/2022_CRC_UP_306_68"
                    ),
  dead_band_manual = c(120,120,65,4),
  stop_offset_secs = c(0,0,0,0))

#apply new incubation period
alldat_trimmed <- alldat_matched %>%
  left_join(time_overrides, by = "Plot") %>%
  group_by(Plot) %>%
  mutate(
    t_start_original = min(TIMESTAMP),
    t_end_original   = max(TIMESTAMP),

    # deadband manual or metadat
    t_start_adjusted = if_else(
      !is.na(dead_band_manual),
```

```
      t_start_original + dead_band_manual,
      t_start_original + as.numeric(coalesce(first(Dead_band), 0))
    ),

    # new offset
    t_end_adjusted = if_else(
      !is.na(stop_offset_secs),
      t_end_original + stop_offset_secs,
      t_end_original
    )
  ) %>%
  filter(TIMESTAMP >= t_start_adjusted,
         TIMESTAMP <= t_end_adjusted) %>%
  ungroup() %>%
  select(-t_start_original, -t_end_original,
         -t_start_adjusted, -t_end_adjusted,
         -dead_band_manual, -stop_offset_secs)

# alldat_trimmed %>%
#     filter(Plot == "8/17/2022_OWC_UP_6_147") %>%
#     ggplot(aes(TIMESTAMP, CH4)) +
#     geom_point() + theme_classic()
```

10 Calculate fluxes

```
# TO Do - NEED TO PULL TEMP from the weather station
#change the units
alldat_trimmed$CH4_nmol <- ffi_ppb_to_nmol(alldat_trimmed$CH4,
                                volume = alldat_trimmed$Volume, # m3
                                temp = alldat_trimmed$Temp)    # degrees C
```

```
Assuming atm = 101325 Pa
```

```
Using R = 8.31446261815324 m3 Pa K-1 mol-1
```

```
#> Assuming atm = 101325 Pa
#> Using R = 8.31446261815324 m3 Pa K-1 mol-1
```

```r
alldat_trimmed$CH4_nmol_m2 <- alldat_trimmed$CH4_nmol / 0.16

fluxes <- ffi_compute_fluxes(alldat_trimmed ,
                             group_column = "Plot",
                             time_column = "TIMESTAMP",
                             gas_column = "CH4_nmol_m2",
                             dead_band = 0)
```

```
NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
```

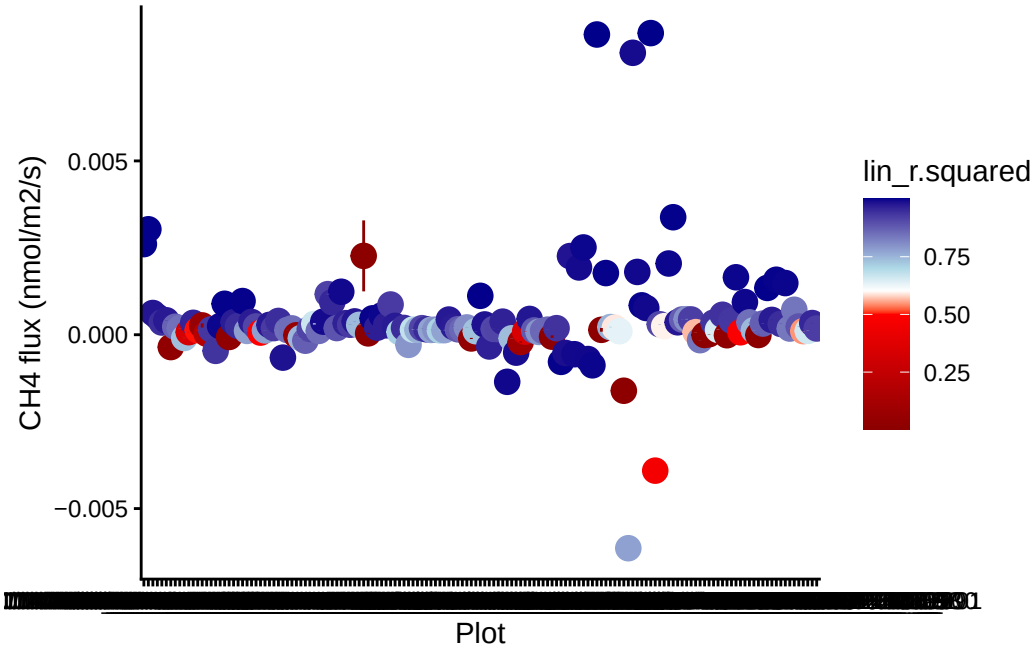
```
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
```

```
head(fluxes)
```

```
                     Plot           TIMESTAMP  HM81_AIC HM81_flux.estimate
1 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29 -1202.306       0.0030555157
2 10/14/2022_CRC_TR_311_72 2022-10-14 10:33:24        NA                 NA
3 10/14/2022_CRC_TR_312_73 2022-10-14 10:26:29        NA                 NA
4 10/14/2022_CRC_TR_313_74 2022-10-14 11:19:44        NA                 NA
5 10/14/2022_CRC_TR_314_75 2022-10-14 10:06:29 -4835.954       0.0005952923
6 10/14/2022_CRC_TR_315_76 2022-10-14 10:13:19 -3449.137       0.0006552876
   HM81_p.value HM81_r.squared  HM81_RMSE   lin_AIC lin_flux.estimate
1 3.072741e-128      0.6603094 0.07919159 -3913.911      0.0026071703
2            NA             NA         NA -3837.760      0.0030329099
3            NA             NA         NA -4151.601      0.0006255273
4            NA             NA         NA -5900.844      0.0005182614
5 2.754526e-254      0.7254439 0.01644391 -6275.945      0.0003517990
6 6.724009e-154      0.6892433 0.01361545 -4862.483      0.0004160704
  lin_flux.std.error lin_int.estimate lin_int.std.error lin_p.value
1       5.326036e-06         71.87489      0.0005511911           0
2       6.354957e-06         70.92464      0.0006943645           0
3       4.273887e-06         72.16874      0.0004423043           0
4       2.849459e-06         70.56655      0.0004429527           0
5       2.843951e-06         71.28601      0.0004913550           0
6       2.964936e-06         71.12060      0.0003410772           0
  lin_r.squared    lin_RMSE  poly_AIC poly_r.squared   poly_RMSE   rob_AIC
1     0.9977598 0.006430965 -4463.569      0.9991965 0.003858741 -3910.921
2     0.9975124 0.008321603 -4130.460      0.9985219 0.006426022 -3835.255
3     0.9755002 0.005160539 -4188.262      0.9772772 0.004979127 -4151.537
4     0.9761571 0.006320845 -6661.371      0.9907222 0.003947813 -5900.707
5     0.9445675 0.007388759 -7089.946      0.9776623 0.004695617 -6275.246
6     0.9705282 0.004193001 -4981.366      0.9759862 0.003791226 -4861.808
  rob_converged rob_flux.estimate rob_flux.std.error    rob_RMSE
1          TRUE      0.0026019366       5.454833e-06 0.006667633
2          TRUE      0.0030411294       6.591054e-06 0.008874780
3          TRUE      0.0006245086       4.154765e-06 0.005032957
4          TRUE      0.0005190243       2.983639e-06 0.007076659
5          TRUE      0.0003516754       2.974804e-06 0.008100417
6          TRUE      0.0004138106       3.004398e-06 0.003903230
       TIMESTAMP_min       TIMESTAMP_max
1 2022-10-14 10:43:00 2022-10-14 10:45:59
2 2022-10-14 10:31:50 2022-10-14 10:34:59
3 2022-10-14 10:25:00 2022-10-14 10:27:59
4 2022-10-14 11:17:30 2022-10-14 11:21:59
```

```
5 2022-10-14 10:04:00 2022-10-14 10:08:59
6 2022-10-14 10:11:40 2022-10-14 10:14:59
```

```
#merging fluxes with initial CH4 concentration
fluxes <- fluxes %>%
  left_join(df_CH4_in,by = "Plot")

#take a look at the fluxes we calculated with R2
ggplot(fluxes, aes(Plot, lin_flux.estimate, color = lin_r.squared)) +
  geom_point(size=4) + theme_classic() +
  geom_linerange(aes(ymin = lin_flux.estimate- lin_flux.std.error,
                     ymax = lin_flux.estimate+ lin_flux.std.error)) +
  scale_color_gradientn(colours = c("darkred",  "red", "white","lightblue", "darkblue"),
                        values = c(0, 0.5, 0.6, 0.7, 1)) +
  ylab("CH4 flux (nmol/m2/s)")
```



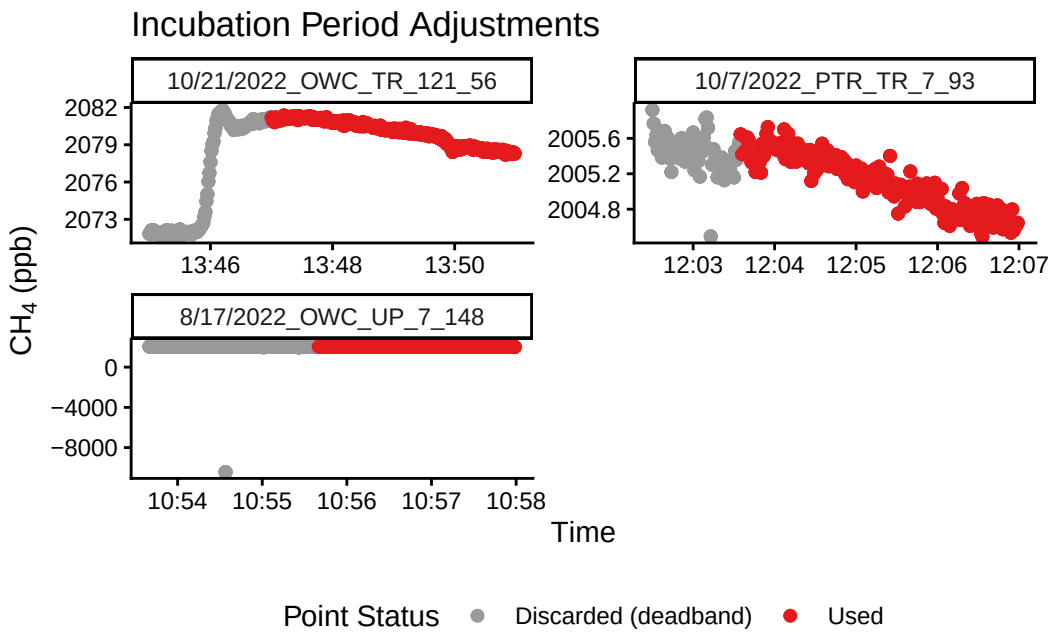

```
#sort fluxes by R2
fluxes_good <- as.data.frame(subset(fluxes, lin_r.squared >= .90, select = Plot:TIMESTAMP_ma
fluxes_lowr2 <- as.data.frame(subset(fluxes, lin_r.squared < .90 & lin_flux.estimate < 1, se
fluxes_weird <- as.data.frame(subset(fluxes, lin_r.squared < .90, select = Plot:TIMESTAMP_ma

#find fluxes that represent the first and fourth quartile of the data
```

```
low <- quantile(fluxes$lin_flux.estimate, 0.05)
high <- quantile(fluxes$lin_flux.estimate, 0.95)

fluxes_low <- as.data.frame(subset(fluxes, lin_flux.estimate <= low, select = Plot:TIMESTAMP
fluxes_high <- as.data.frame(subset(fluxes, lin_flux.estimate >= high, select = Plot:TIMESTA

tails <- rbind(fluxes_low, fluxes_high)
```

11 check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)

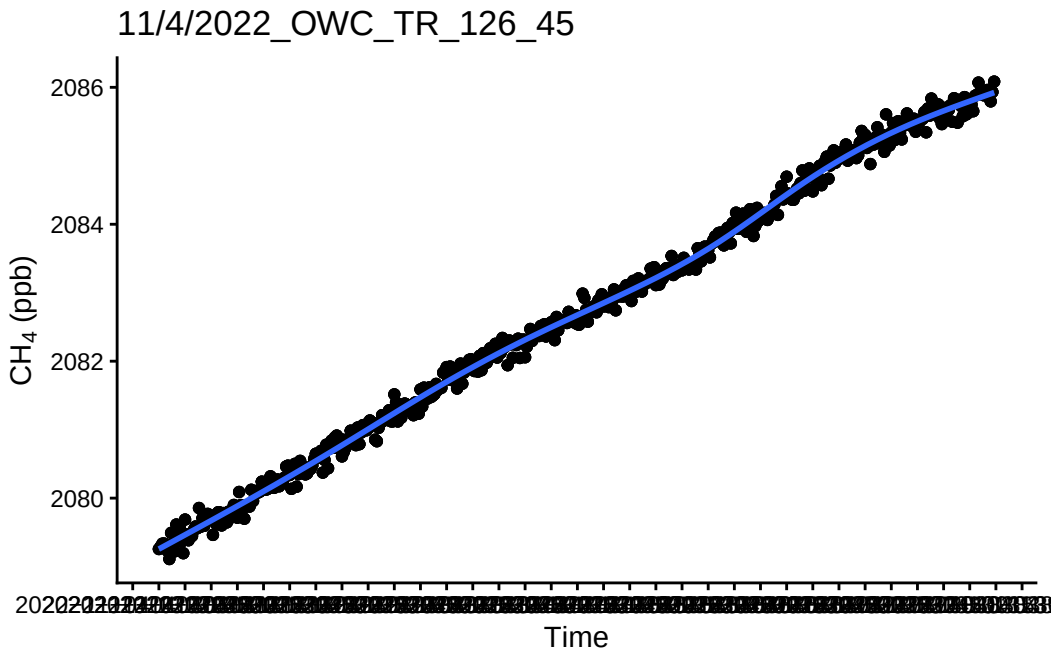
```
# grab trim limits for each overridden Plot
trim_limits <- alldat_trimmed %>%
  filter(Plot %in% time_overrides$Plot) %>%
  group_by(Plot) %>%
  summarise(
    t_start_trim = min(TIMESTAMP),
    t_end_trim   = max(TIMESTAMP) )

# use full alldat_matched + trim limits to classify each point
r_all <- alldat_matched %>%
  filter(Plot %in% time_overrides$Plot) %>%
  left_join(trim_limits, by = "Plot") %>%
  group_by(Plot) %>%
  mutate(
    time_sec = as.numeric(difftime(TIMESTAMP, min(TIMESTAMP), units = "secs")),
    point_status = case_when(
      TIMESTAMP < t_start_trim ~ "Discarded (deadband)",
      TIMESTAMP > t_end_trim   ~ "Discarded (stop trim)",
      TRUE                     ~ "Used"  )) %>%
  ungroup()

# plot all overridden plots
ggplot(r_all, aes(x = TIMESTAMP, y = CH4, color = point_status)) +
  geom_point(size = 1.8) +
  facet_wrap(~ Plot, scales = "free", ncol = 2) +
  scale_color_manual(
    values = c(
      "Used"                  = "#E41A1C",
```

```
    "Discarded (deadband)"  = "grey60",
    "Discarded (stop trim)" = "grey20"
  )
) +
theme_classic() +
ylab(expression(paste(CH[4], " (ppb)"))) +
xlab("Time") +
labs(
  title = "Incubation Period Adjustments",
  color = "Point Status") +
theme(legend.position = "bottom")
```

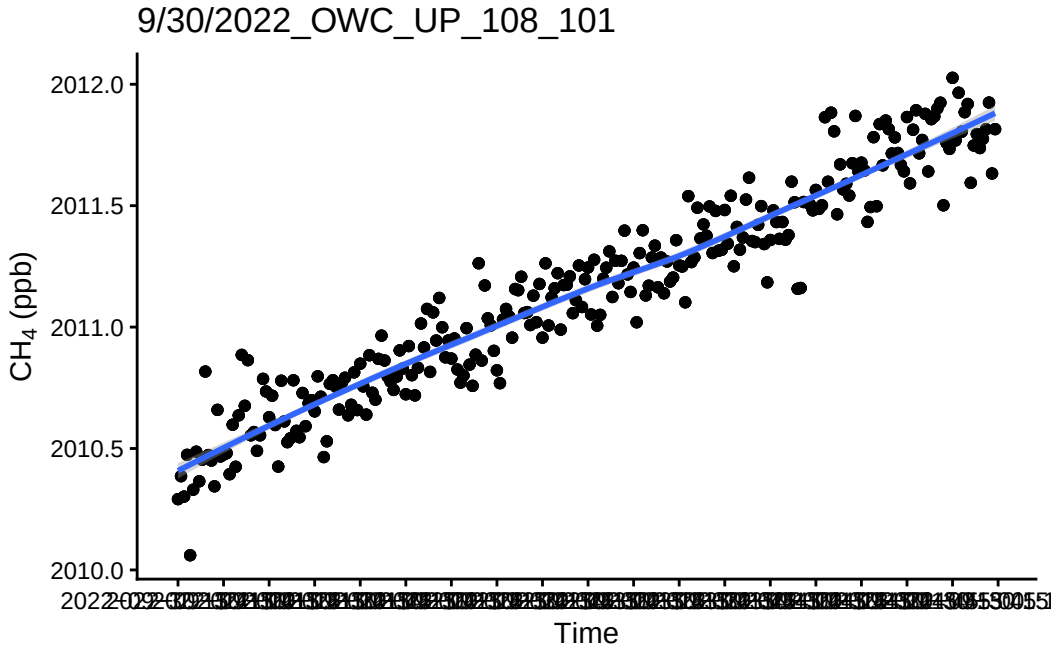


12 Spot check a few fluxes with R2 $>$ 0.9

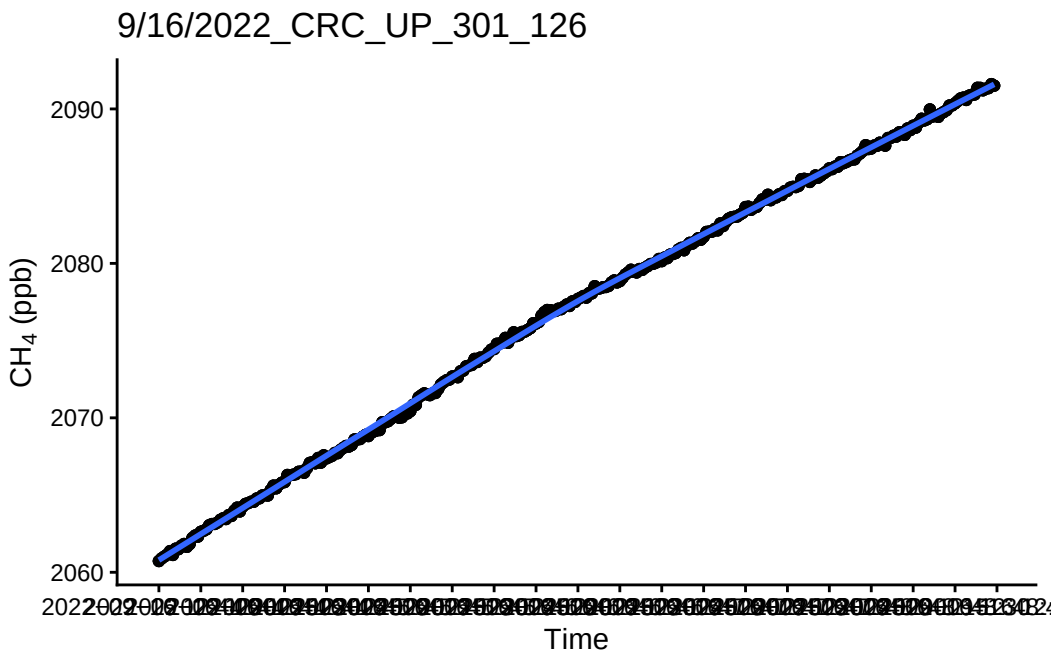
```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



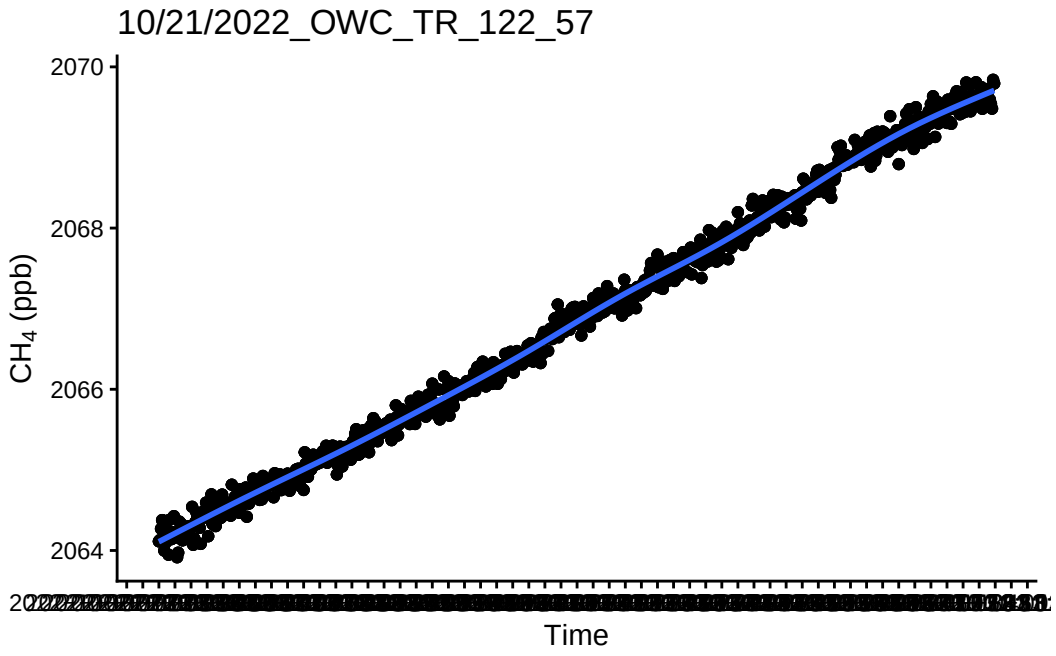

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



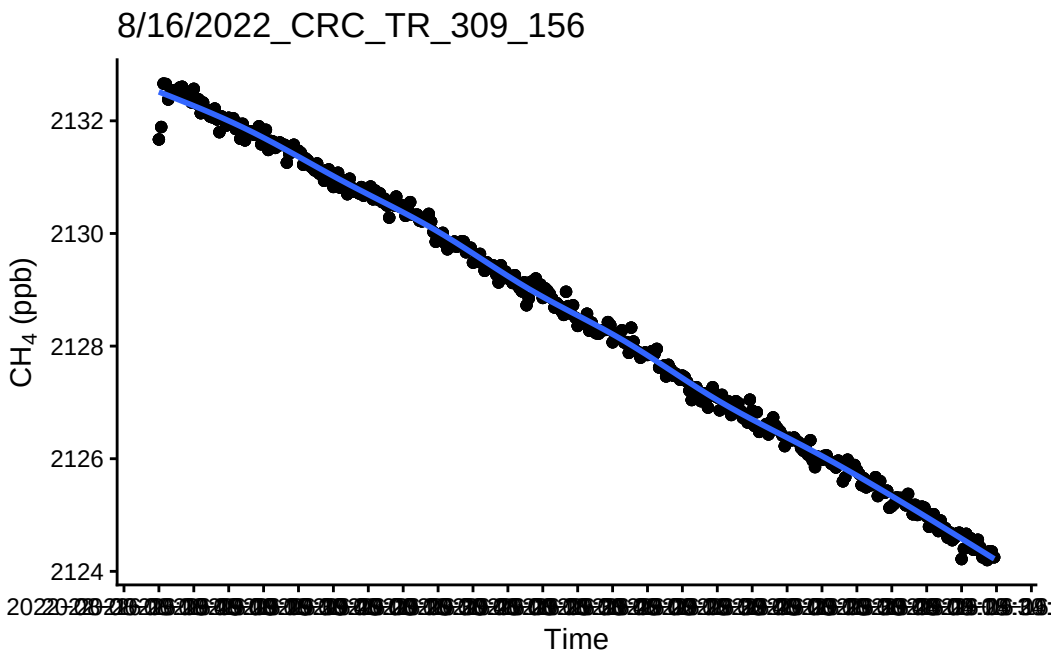

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



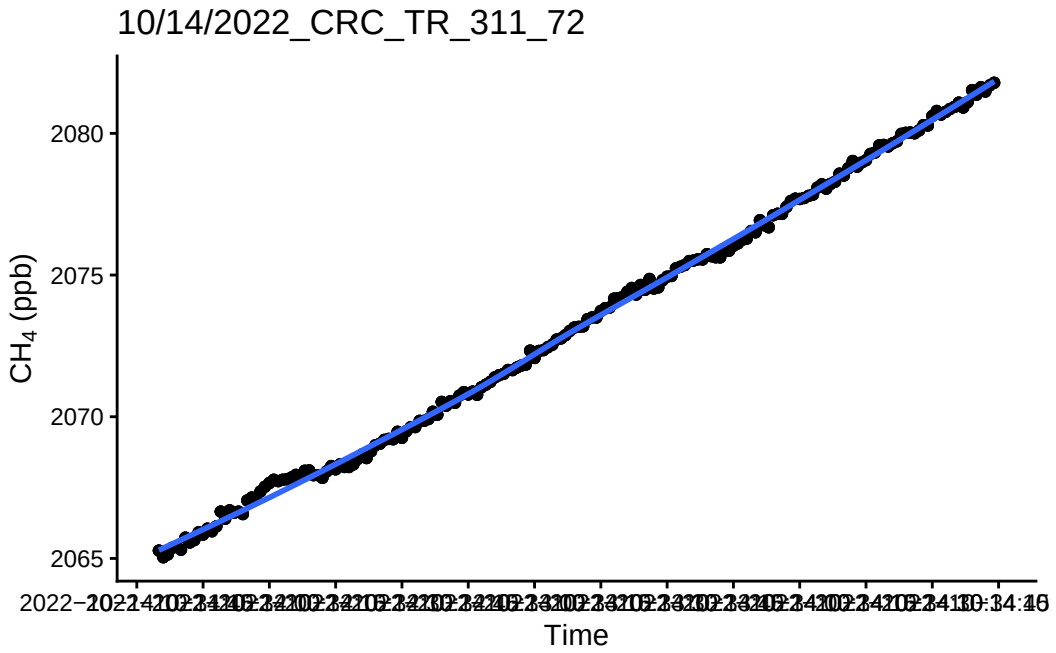

```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



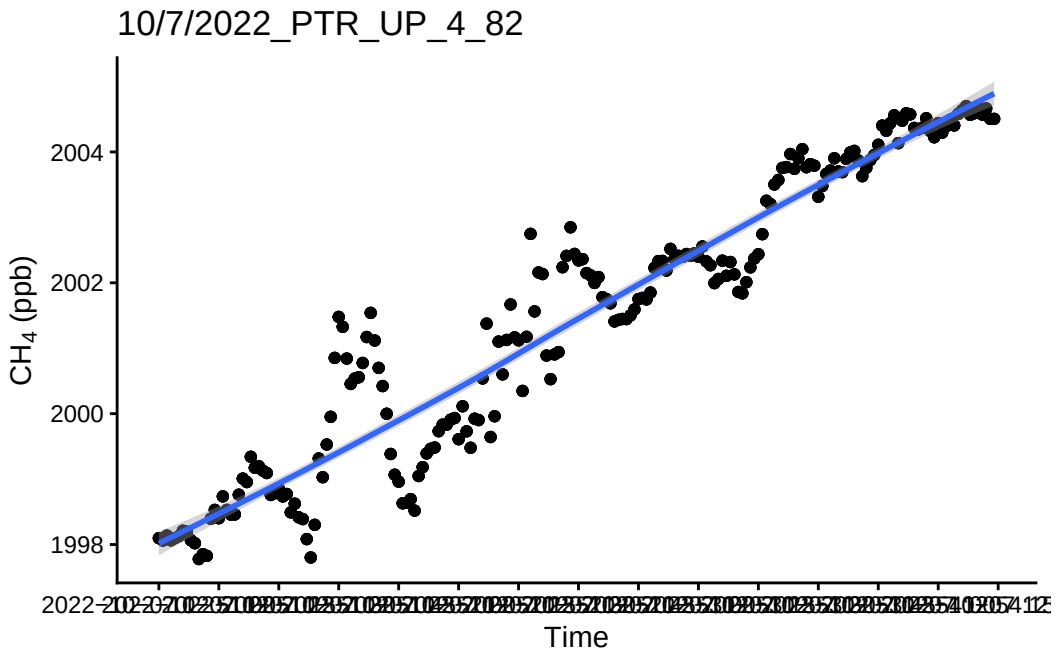

```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



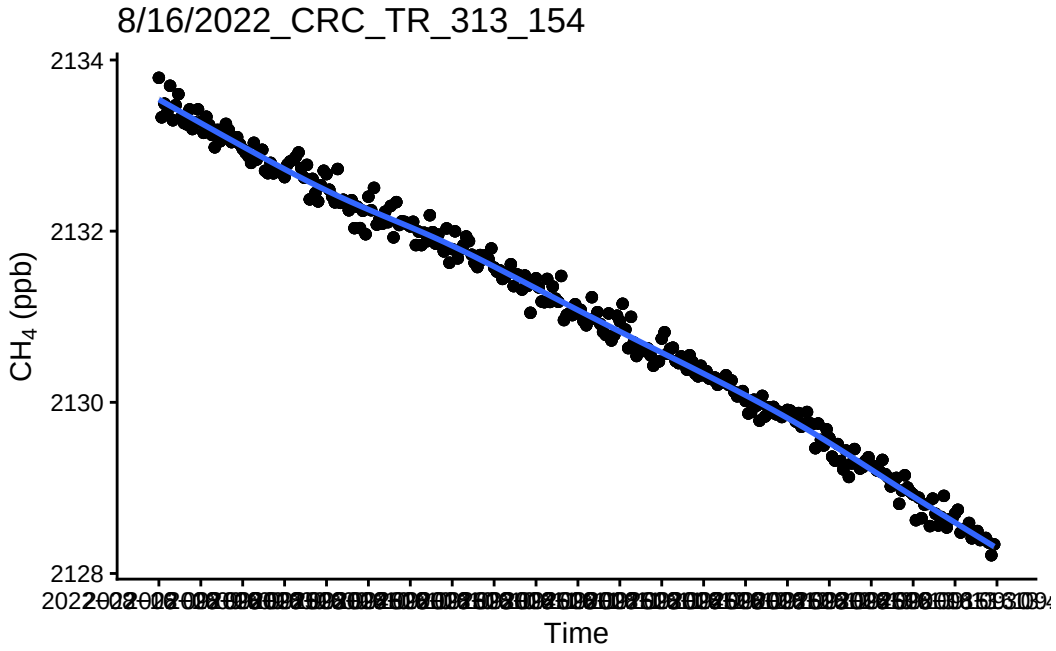

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



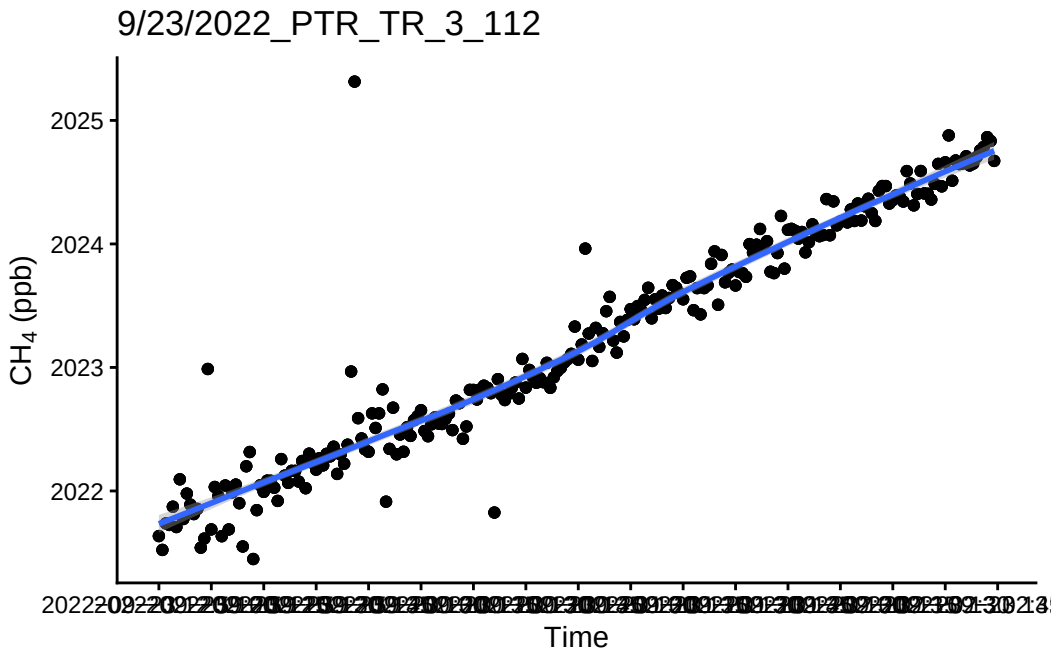

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```

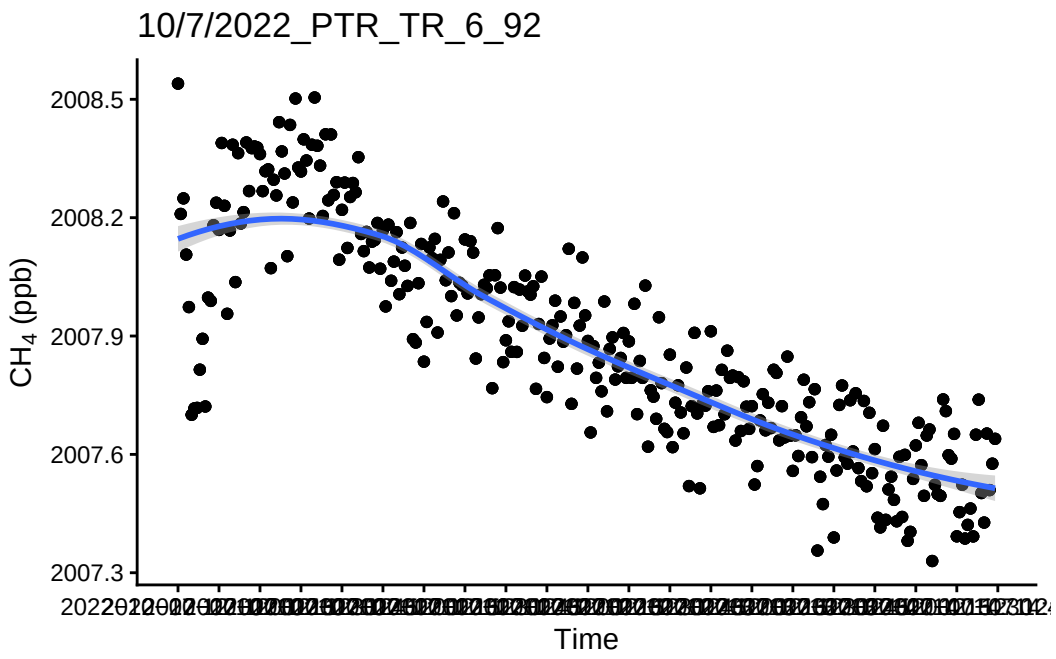



```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

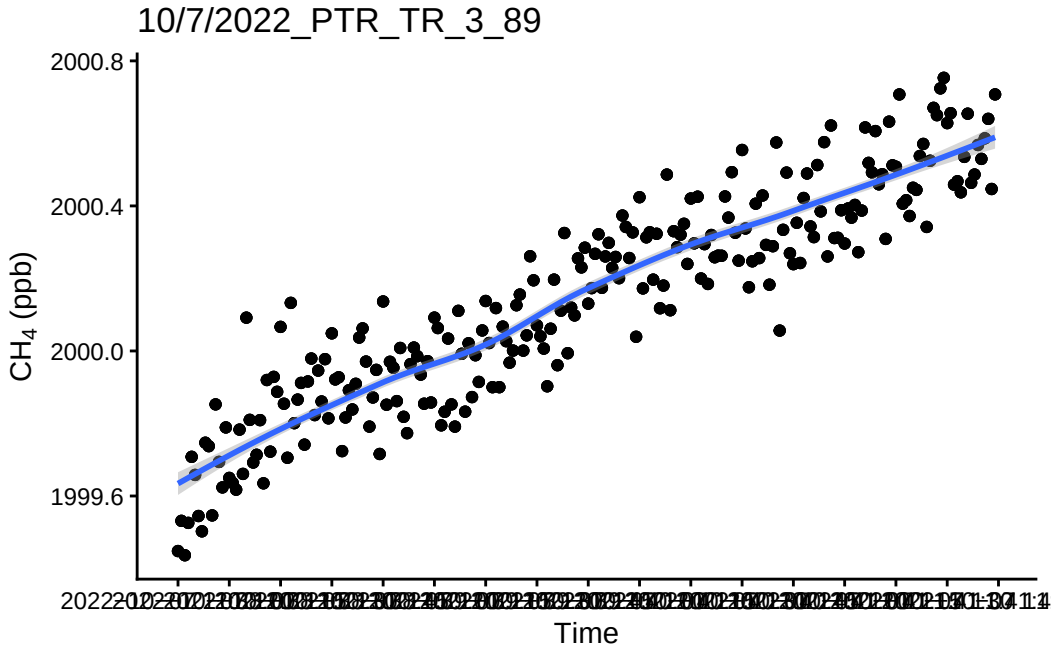


13 Take a look at the fluxes <0.9 and a flux estimate below 1

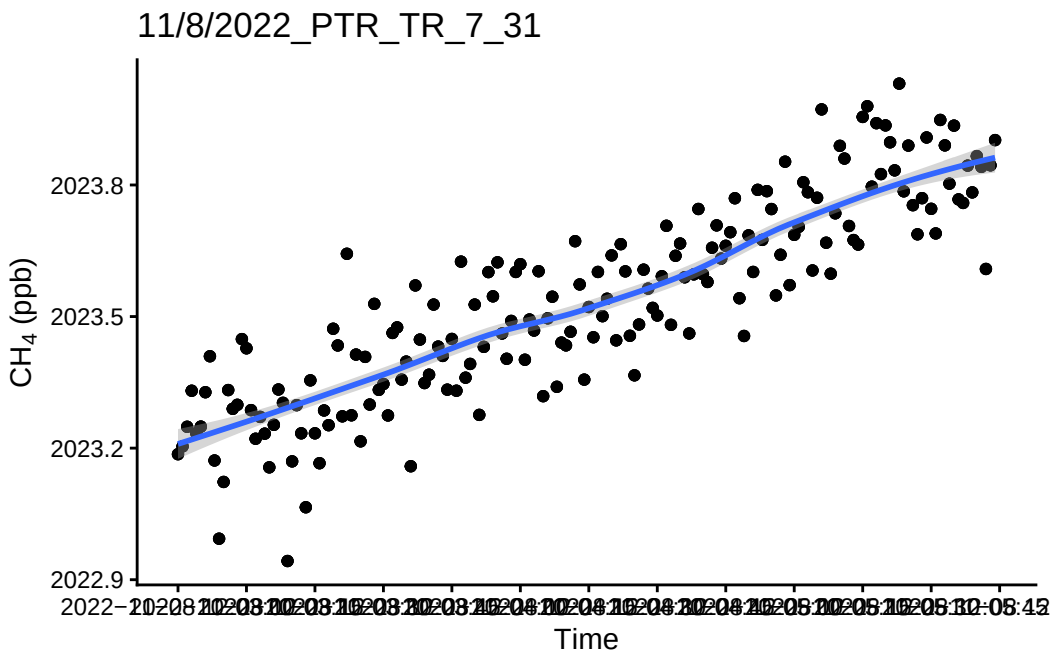
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



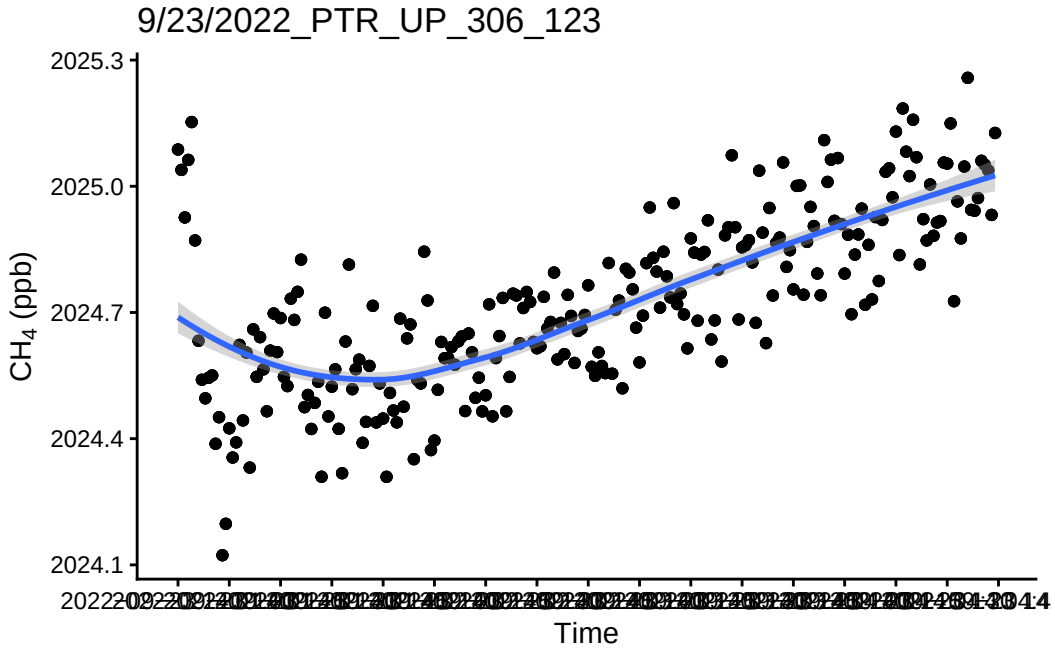

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



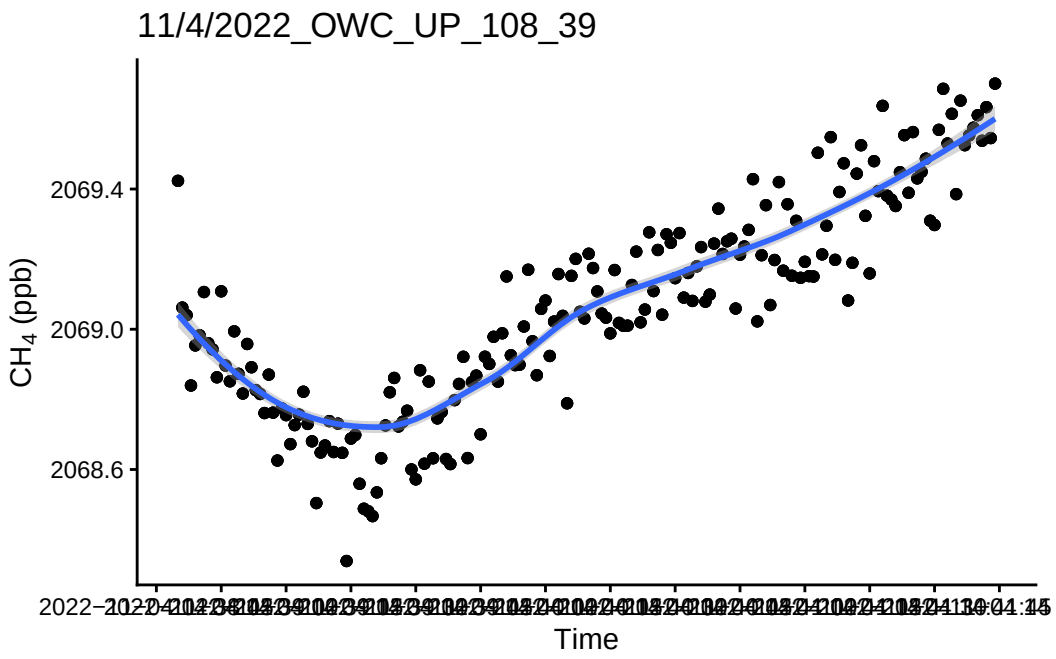

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



`geom_smooth()` using method = 'loess' and formula = 'y ~ x'



`geom_smooth()` using method = 'loess' and formula = 'y ~ x'



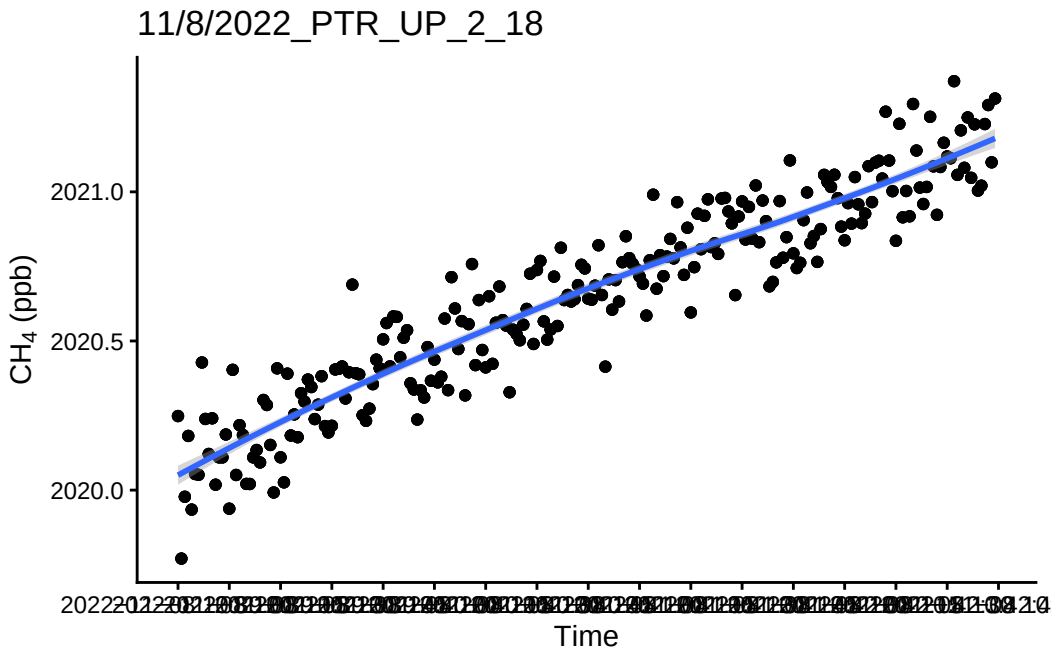
14 Take a look at the bad fluxes <0.9 and have a high flux estimate

15 See if we want to remove these fluxes or if we want to try and recalculate these ones?

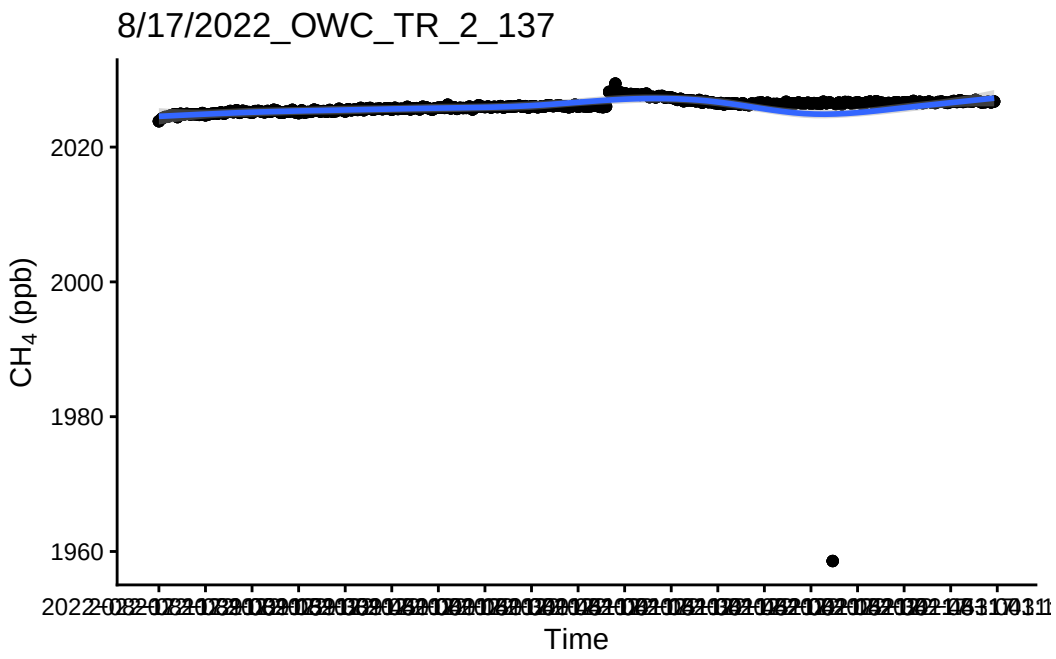
```
### pull out fluxes with R2 < 0.9 and put them in a list
bad_plots <- unique(fluxes_weird$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
bad_sample <- sample(bad_plots, 5, replace=FALSE)

#Loop to run five random plots subset above
for(P in bad_sample){
  r1 <- subset(alldat_trimmed, Plot == P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
  scale_x_datetime(breaks = "15 sec") +
  ylab(expression(paste( CH [4], " (ppb)"))) +
```

```
  xlab("Time") + labs(title = P) +
  geom_smooth()
  print(p1)
  }
```

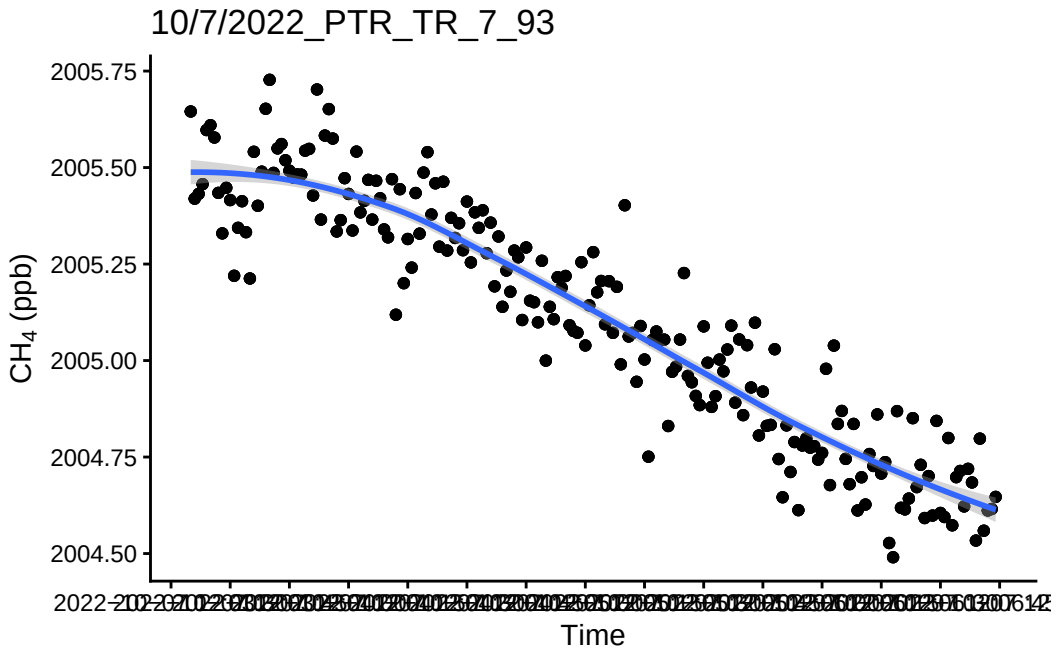
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



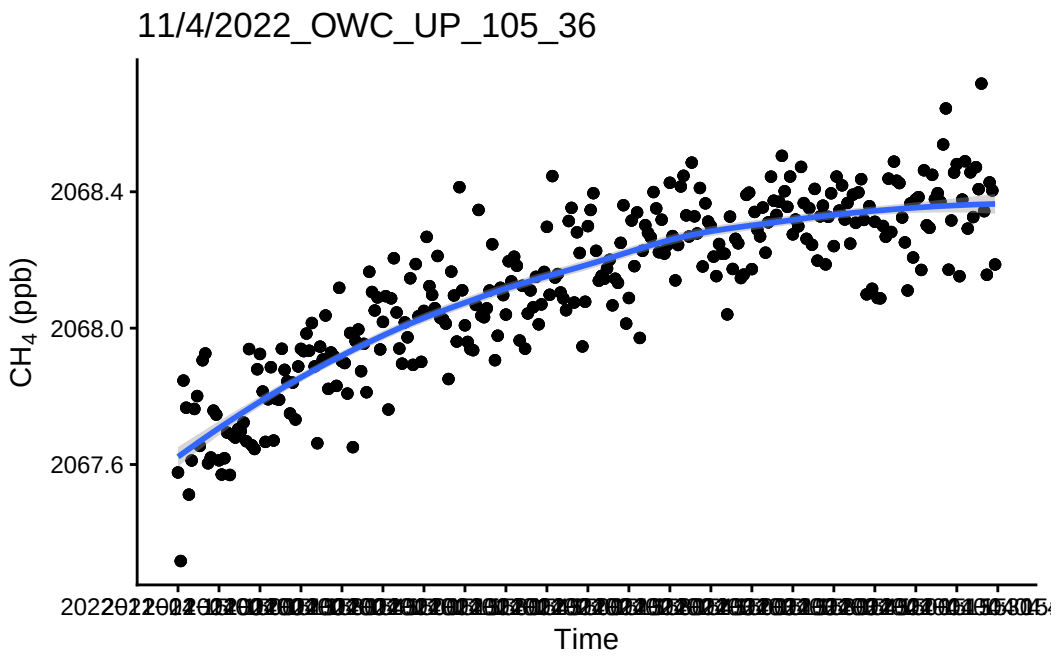

```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



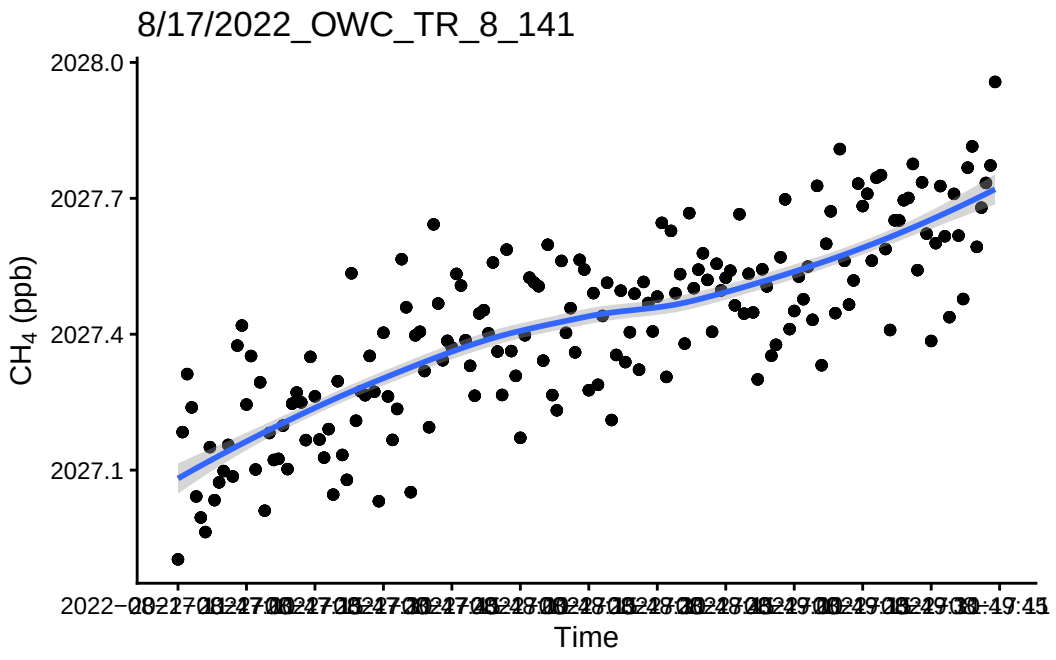

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

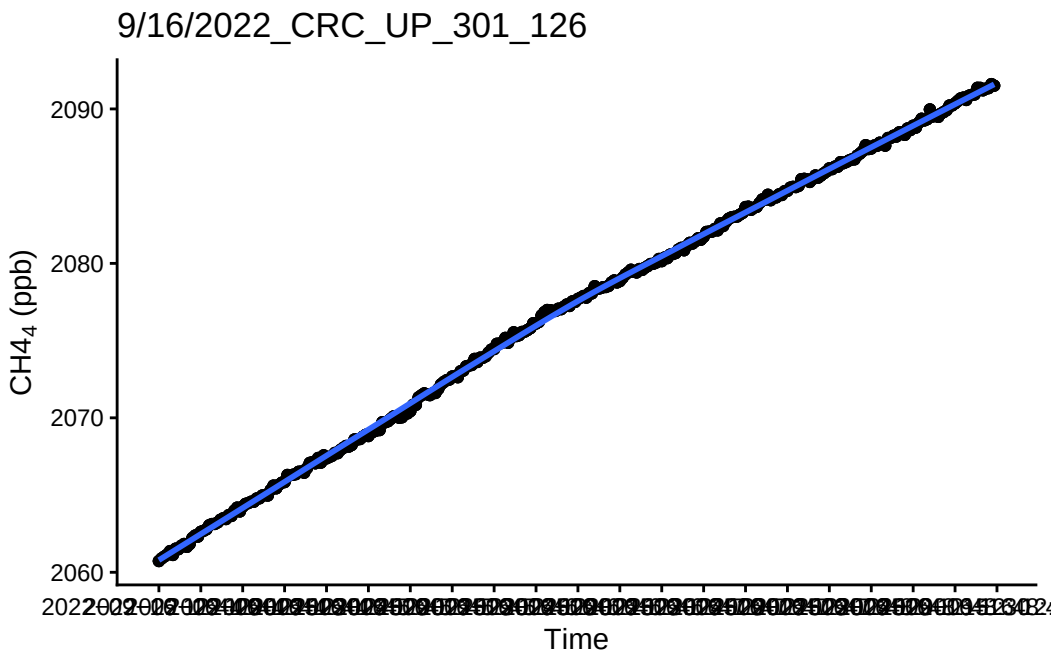


```
####################
```

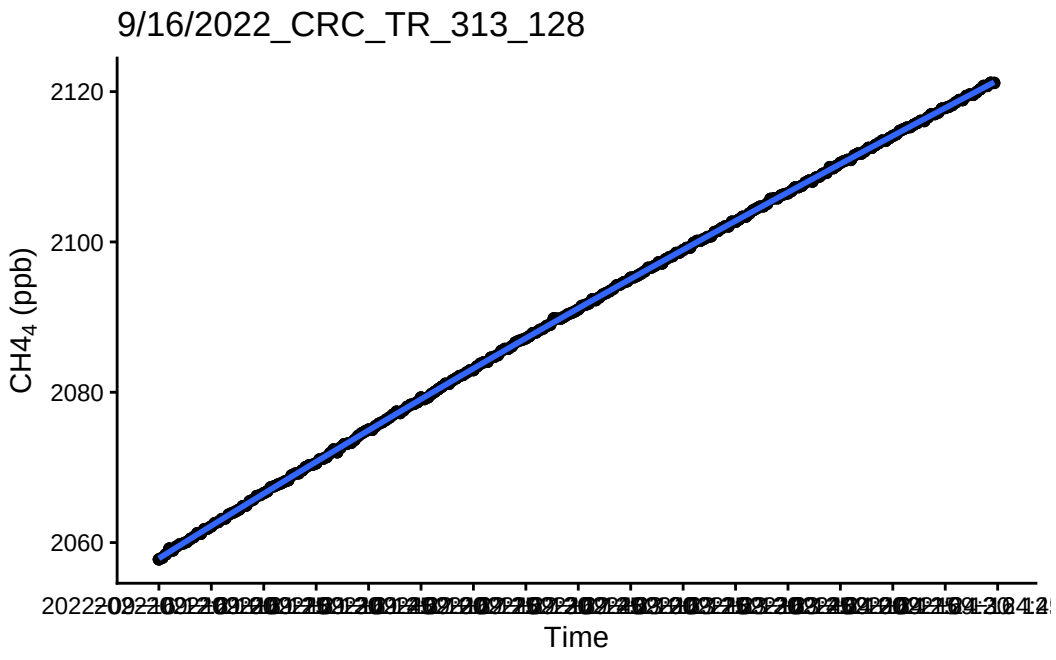
16 Look at the really high or really low ones to see if there is a timing issue or something

```
### pull out fluxes in the 5% and 95% tails to look at really high and really low
tail_plots <- unique(tails$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
tail_sample <- sample(tail_plots, 2, replace=FALSE)

#Loop to run five random plots subset above
for(P in tail_sample){
  r1 <- subset(alldat_trimmed, Plot == P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
  scale_x_datetime(breaks = "15 sec") +
  ylab(expression(paste( CH4 [4], " (ppb)"))) +
  xlab("Time") + labs(title = P) +
  geom_smooth()
  print(p1)
  }
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



`geom_smooth()` using method = 'loess' and formula = 'y ~ x'

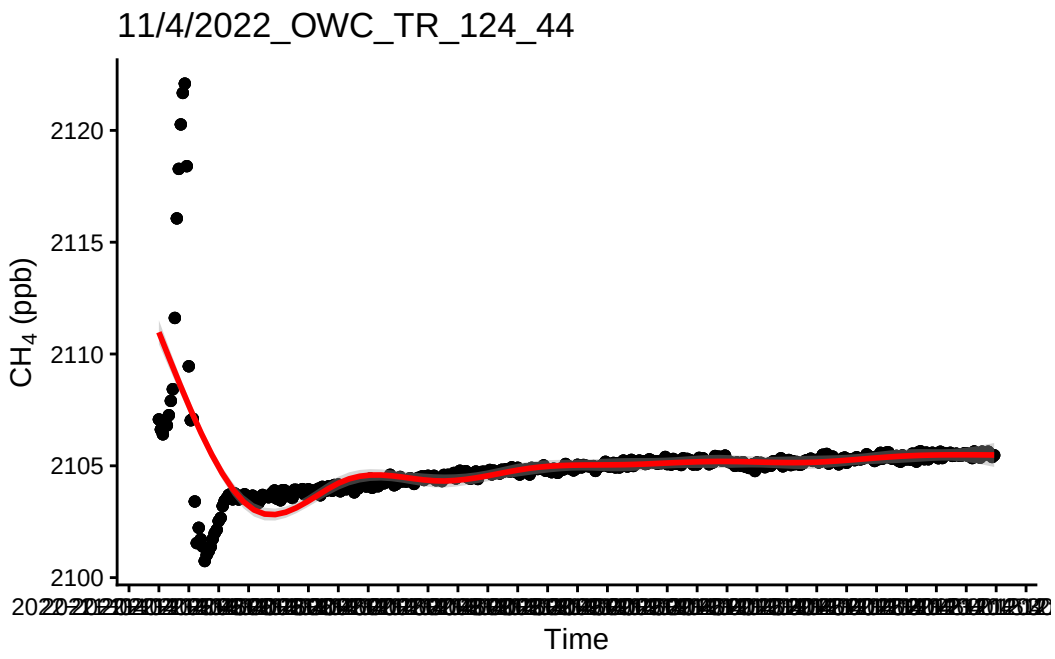


```
#would be nice to put on the plot the flux estimate and if it is a high or low or something
####################
```

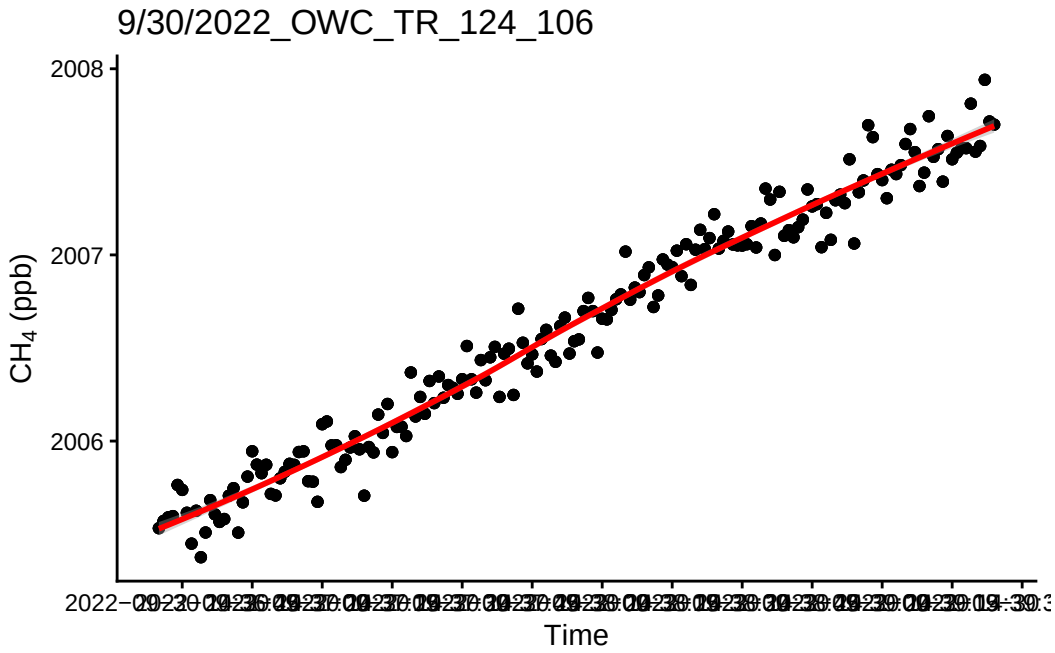
17 Check all fluxes

```
##Put all fluxes together
fluxes_all <- as.data.frame(subset(fluxes))

all_plots <- unique(fluxes_all$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
all_sample <- sample(all_plots, 5, replace=FALSE)

#Loop to run five random plots subset above
for(P in all_sample){
  r1 <- subset(alldat_trimmed, Plot== P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
  scale_x_datetime(breaks = "15 sec") +
  ylab(expression(paste( CH [4], " (ppb)"))) +
  xlab("Time") + labs(title = P) +
  geom_smooth(color="red")
  print(p1)
  }
```

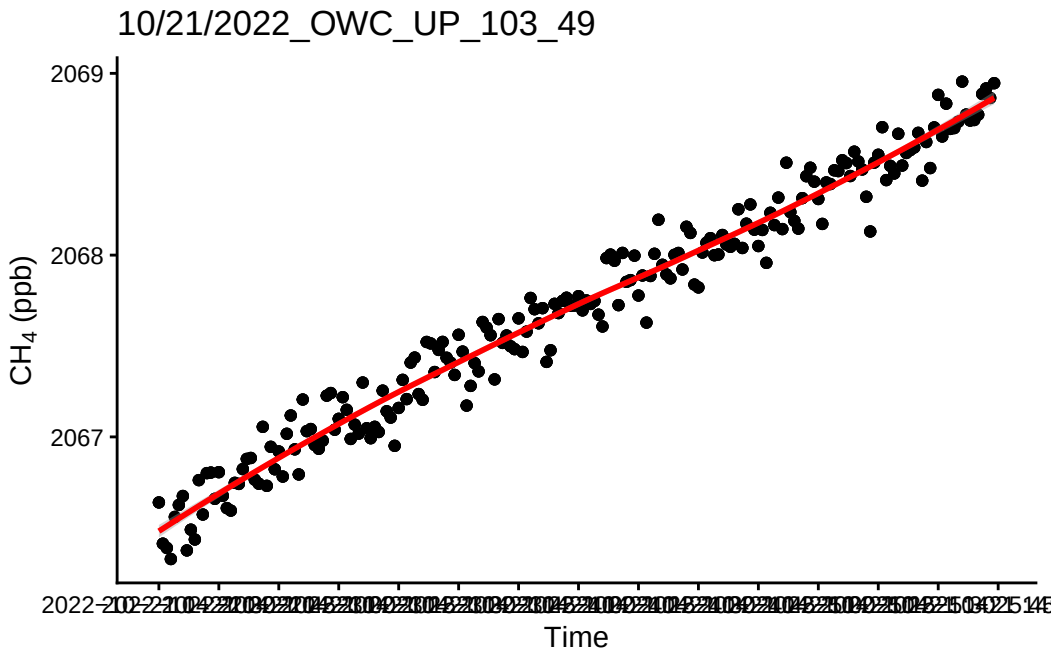
```
`geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



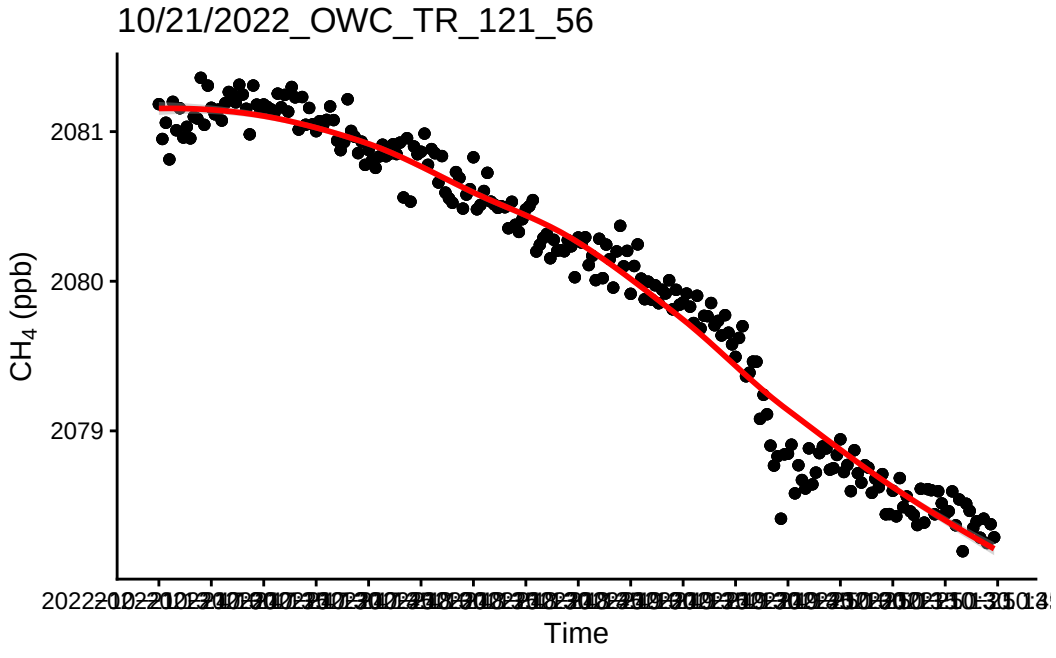

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



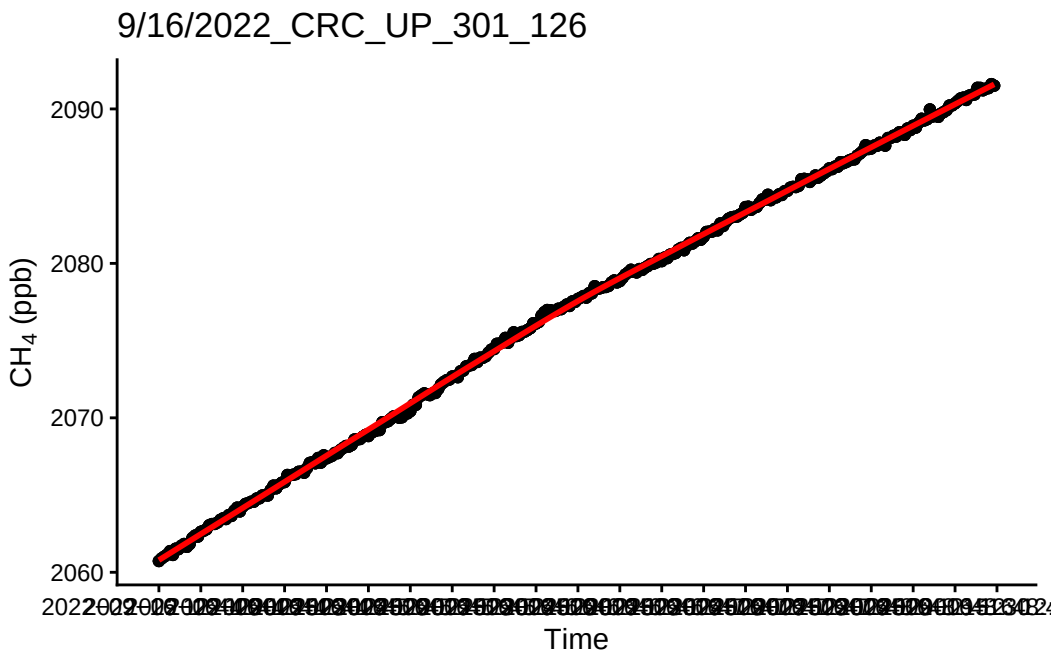

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
####################
```

18 QAQC flags

```
# QAQC flags
fluxes_flagged <- fluxes %>%
  mutate(
    # model fit flags
    CH4_lin_pass  = (lin_r.squared  >= 0.85 | lin_RMSE  < 5) & !is.na(lin_r.squared),
    CH4_HM81_pass = (HM81_r.squared >= 0.85 | HM81_RMSE < 5) & !is.na(HM81_r.squared),
    CH4_poly_pass = (poly_r.squared >= 0.85 | poly_RMSE < 5) & !is.na(poly_r.squared),
    # negative flux flag (CH4 negative fluxes may be valid, but flagging anyway)
    CH4_neg_flux        = lin_flux.estimate < 0 | is.na(lin_flux.estimate),
    # rejection flags
    CH4_rejected_model  = !(CH4_lin_pass | CH4_HM81_pass | CH4_poly_pass)
  )

r2_summary <- fluxes_flagged %>%
  summarise(
    CH4_total_obs          = n(),
```

```
    CH4_n_rejected_model   = sum(CH4_rejected_model,    na.rm = TRUE),
    CH4_pct_rejected_total = 100 * CH4_n_rejected_model / CH4_total_obs,
    CH4_n_lin_pass         = sum(CH4_lin_pass,  na.rm = TRUE),
    CH4_n_HM81_pass        = sum(CH4_HM81_pass, na.rm = TRUE),
    CH4_n_poly_pass        = sum(CH4_poly_pass, na.rm = TRUE)
  )

# formatted table for PDF
r2_summary %>%
  tidyr::pivot_longer(everything(), names_to = "Metric", values_to = "Value") %>%
  mutate(
    Metric = recode(Metric,
      CH4_total_obs          = "Total observations",
      CH4_n_rejected_model   = "Rejected - poor model fit (n)",
      CH4_pct_rejected_total = "Rejected - total (%)",
      CH4_n_lin_pass         = "Passed - Linear model (n)",
      CH4_n_HM81_pass        = "Passed - HM81 model (n)",
      CH4_n_poly_pass        = "Passed - Polynomial model (n)"
    ),
    Value = ifelse(Metric == "Rejected - total (%)",
                   round(Value, 1),
                   as.integer(Value))
  ) %>%
  kable(
    caption  = "QA/QC CH4 Flux Summary",
    align    = c("l", "r"),
    booktabs = TRUE,
    format   = "latex"
  ) %>%
  kable_styling(
    latex_options = c("striped", "hold_position"),
    full_width    = FALSE
  ) %>%
  pack_rows("Rejections",        2, 3) %>%
  pack_rows("Model performance", 4, 6)
```

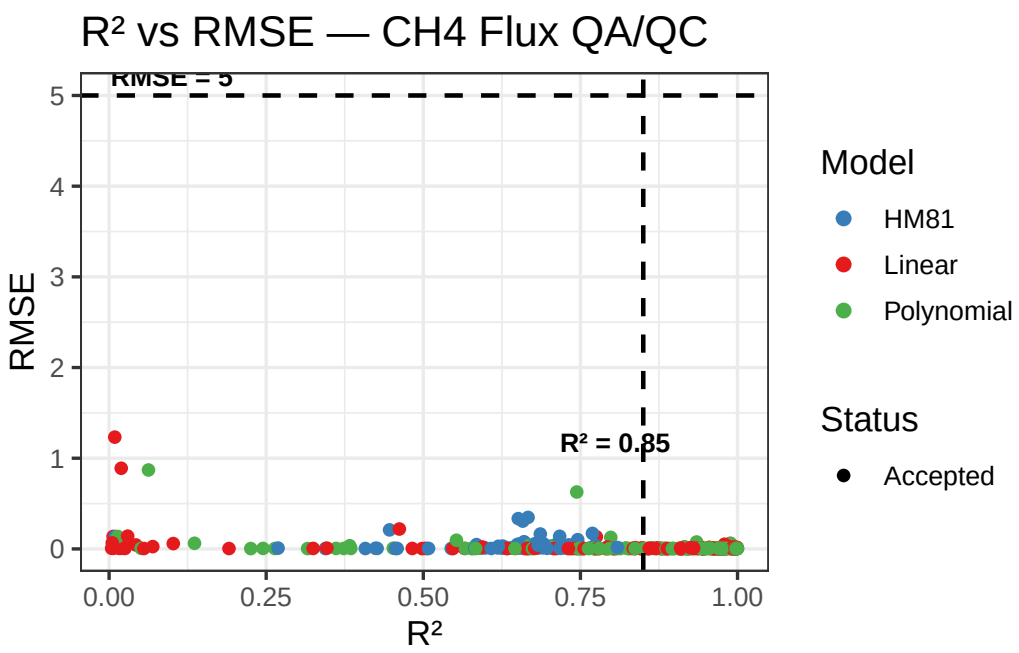
```
# R² vs RMSE diagnostic plot
set.seed(42)
plot_data <- fluxes_flagged %>%
  slice_sample(n = min(15000, nrow(.))) %>%
  select(
    lin_r.squared,  lin_RMSE,  CH4_lin_pass,
```

Table 1: QA/QC CH4 Flux Summary

Metric	Value
Total observations	151
Rejections	
Rejected — poor model fit (n)	0
Rejected — total (%)	0
Model performance	
Passed — Linear model (n)	151
Passed — HM81 model (n)	75
Passed — Polynomial model (n)	151

```
    HM81_r.squared, HM81_RMSE, CH4_HM81_pass,
    poly_r.squared, poly_RMSE, CH4_poly_pass
  ) %>%
  rename(
    lin_pass  = CH4_lin_pass,
    HM81_pass = CH4_HM81_pass,
    poly_pass = CH4_poly_pass
  ) %>%
  pivot_longer(cols = everything(),
    names_to      = c("model", ".value"),
    names_pattern = "(lin|HM81|poly)_(.*)"
  ) %>%
  rename(r2 = r.squared) %>%
  mutate(
    model  = recode(model, lin = "Linear", HM81 = "HM81", poly = "Polynomial"),
    status = case_when(
      pass                ~ "Accepted",
      TRUE                ~ "Rejected"
    )
  ) %>%
  filter(!is.na(r2), !is.na(RMSE))

QAQC_plot <- ggplot(plot_data, aes(x = r2, y = RMSE, color = model, shape = status)) +
  geom_point(size = 2) +
  geom_vline(xintercept = 0.85, linetype = "dashed", color = "black", linewidth = 0.8) +
  geom_hline(yintercept = 5,    linetype = "dashed", color = "black", linewidth = 0.8) +
  annotate("text", x = 0.7, y = max(plot_data$RMSE, na.rm = TRUE) * 0.95,
           label = "R² = 0.85", hjust = -0.1, size = 3.5, fontface = "bold") +
```

```
  annotate("text", x = 0.1, y = 5,
           label = "RMSE = 5", vjust = -0.5, size = 3.5, fontface = "bold") +
  scale_color_manual(values = c(
    "Linear"     = "#E41A1C",
    "HM81"       = "#377EB8",
    "Polynomial" = "#4DAF4A"
  )) +
  scale_shape_manual(values = c(
    "Accepted"     = 16,
    "Rejected"     = 4
  )) +
  labs(
    title = "R² vs RMSE - CH4 Flux QA/QC",
    x     = "R²",
    y     = "RMSE",
    color = "Model",
    shape = "Status"
  ) +
  theme_bw(base_size = 13) +
  theme(legend.position = "right")

print(QAQC_plot)
```

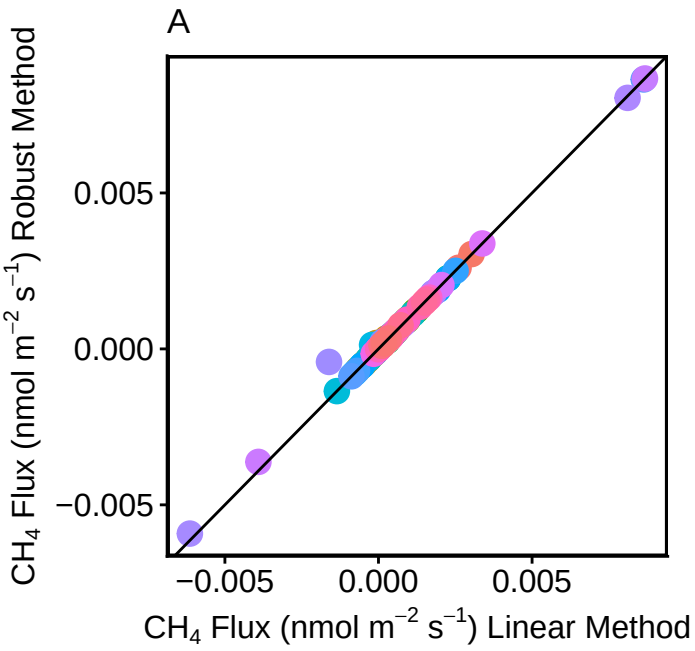


19 Adding manual flag to reject any fluxes that do not fit criteria

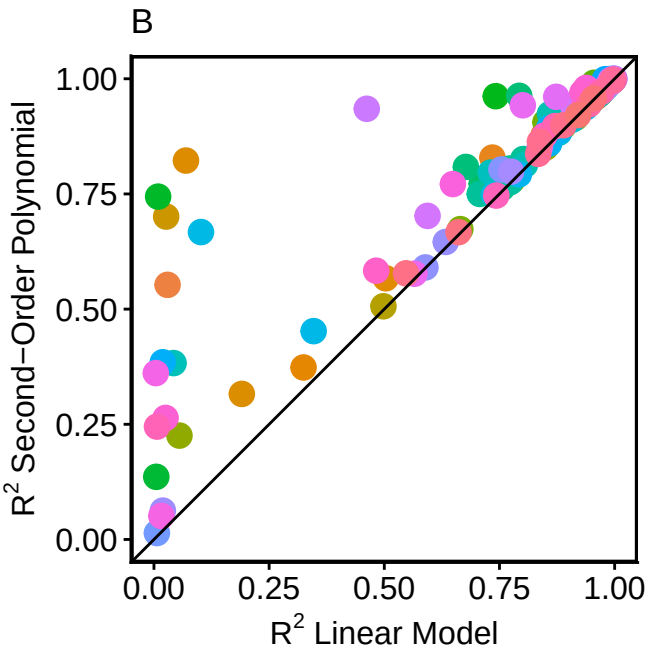
```
# adding manual flags to the ones that are crazy off based on visual checks
manual_reject <- c()

fluxes_flagged <- fluxes_flagged %>%
  mutate(
    CH4_rejected_manual  = Plot %in% manual_reject,
    CH4_flux_rejected   = CH4_rejected_model |CH4_rejected_manual)

# fluxes_flagged %>%
#   filter(CO2_rejected_manual) %>%
#   select(Plot, lin_r.squared, lin_RMSE, CH4_rejected_model, CH4_rejected_manual, CH4_flux_
```

20 Do some visualization of QAQC plots to assess fluxes

```
allflux_filtered <-fluxes_flagged%>%filter(!CH4_rejected_model) #CHANGE FOR CH4_flux_rejecte

p1 <- ggplot(allflux_filtered, aes(lin_flux.estimate, rob_flux.estimate, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  labs(title = "A") +
  ylab(expression(paste( CH [4], " Flux (nmol m"^-2* " s"^-1*") Robust Method"))) +
  xlab(expression(paste( CH [4], " Flux (nmol m"^-2* " s"^-1*") Linear Method"))) +
  guides(color = guide_legend(title = "Plot")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
p1
```

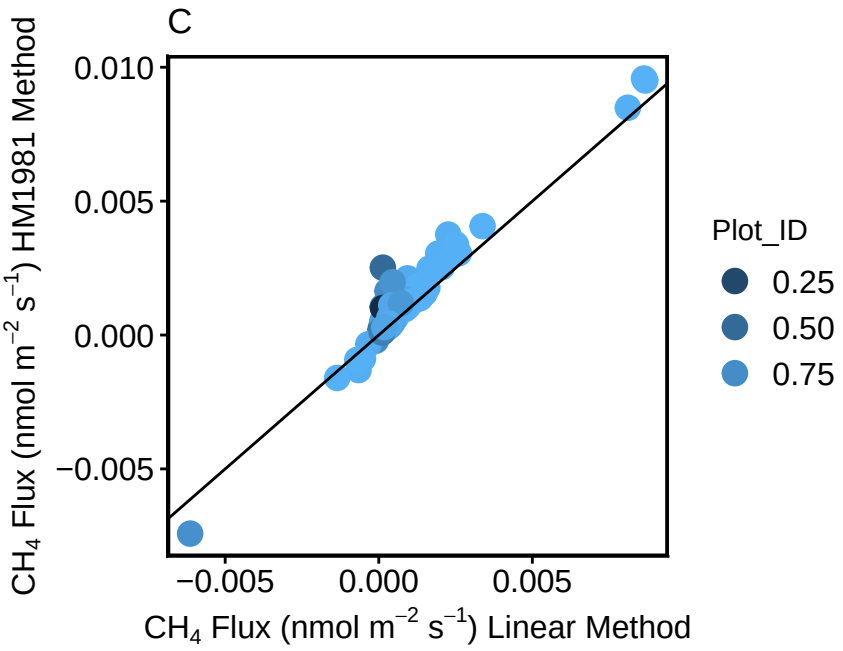



```
p2 <- ggplot(allflux_filtered, aes(lin_r.squared, poly_r.squared, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  xlim(0,1) + ylim(0,1) +
  labs(title = "B") +
  ylab(expression(paste("R"^2* " Second-Order Polynomial"))) +
  xlab(expression(paste("R"^2* " Linear Model"))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
p2
```




```
p3 <- ggplot(allflux_filtered, aes(lin_flux.estimate, HM81_flux.estimate, color = lin_r.squa
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  #xlim(0,50) + #ylim(0,50) +
  labs(title = "C") +
  ylab(expression(paste( CH [4], " Flux (nmol m"^-2* " s"^-1*") HM1981 Method"))) +
  xlab(expression(paste( CH [4], " Flux (nmol m"^-2* " s"^-1*") Linear Method"))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="right") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)

p3
```

```
Warning: Removed 76 rows containing missing values or values outside the scale range
(`geom_point()`).
```




```
#ggarrange(p1, p2, p3, ncol=3, nrow=1, common.legend = TRUE, legend="none")
```

21 Read out final dataframe and export for use elsewhere

```
allflux_filtered_renamed <- fluxes_flagged%>%
  mutate(
    Date = mdy(str_extract(Plot, "^[0-9]+/[0-9]+/[0-9]{4}")),
    Month = month.abb[month(Date)],
    Site = str_extract(Plot, "(PTR|OWC|CRC)"),
    Zone = str_extract(Plot, "_TR_|_UP_") %>%
      str_remove_all("_"),
    Replicate = as.integer(
      str_extract(Plot, "(?<=_(TR|UP)_)[0-9]+")
    ),
   Row_ID = as.integer(
      str_extract(Plot, "[0-9]+$")) ) %>%

  # Optional: rebuild a clean Plot label
  mutate(
    Plot_clean = paste(
```

```
      format(Date, "%Y-%m-%d"),
      Site,
      Zone,
      Replicate,
      Row_ID,
      sep = "_"
    )
  ) %>%

  select(
    Plot_clean,
    Date,
    Month,
    Site,
    Zone,
    Replicate,
    Row_ID,
    everything(),
    -Plot
  )



#export as a csv to the folder that you want
write.csv(allflux_filtered_renamed, file="Processed Data/COMPASS_Synoptic_LE_TreeStem_Fluxes_
# #Change file name
```

22